

Метод Кластеризации OPTICS

1. Ключевые понятия метода OPTICS

- **Отличие от DBSCAN:** OPTICS не требует жесткого задания радиуса `eps`. Вместо одного значения он работает в диапазоне (от 0 до `max_eps`) и строит **график достижимости**.
- **Атрибуты после обучения:**
 - `ordering_`: Порядок обработки точек (индексы).
 - `reachability_`: Расстояние достижимости для каждой точки (расстояние до ближайшей точки в кластере). Большие значения указывают на границы кластеров или шум.
- **Извлечение кластеров:**
 1. **Аналог DBSCAN:** Метод `cluster_optics_dbscan` позволяет "обрезать" график на любом уровне `eps`, получая результат, аналогичный DBSCAN.
 2. **Извлечение по умолчанию (`xi`):** Анализирует крутизну склонов графика достижимости, чтобы найти кластеры переменной плотности.
- **Сравнение с DBSCAN:**
 - Результаты похожи, но не идентичны (особенно маркировка граничных точек).
 - Для одного запуска DBSCAN быстрее. Для множества экспериментов с разными `eps` — OPTICS эффективнее.
- **Сложность:** OPTICS экономит память (масштабируется как $O(n)$, а не $O(n^2)$), но может быть медленнее HDBSCAN.

2. Программная реализация на Python

Для демонстрации создадим синтетические данные, обучим модель OPTICS и визуализируем график достижимости. **Установка и импорт**

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.cluster import OPTICS
```

```
from sklearn.datasets import make_blobs
```

```
# Генерация данных: три кластера разной плотности
```

```
X, y = make_blobs(n_samples=300, centers=3, cluster_std=[0.5, 1.0, 1.5],
random_state=42)
```

Обучение модели и визуализация

```
# Создание и обучение модели OPTICS
```

```
# max_eps = inf (по умолчанию) - ищем кластеры любой плотности
```

```
# min_samples = 10 - минимальное количество точек для формирования кластера
```

```
clust = OPTICS(min_samples=10, max_eps=np.inf, metric='minkowski', p=2)
```

```
clust.fit(X)
```

```
# Извлечение результатов
```

```
labels = clust.labels_ # Метки кластеров (-1 - шум)
```

```
reachability = clust.reachability_[clust.ordering_] # Расстояния достижимости в порядке обработки
```

```
ordering = clust.ordering_
```

```
# --- Визуализация 1: Исходные точки с цветами кластеров ---
```

```
plt.figure(figsize=(12, 5))
```

```
plt.subplot(1, 2, 1)
```

```
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='rainbow', s=30)
```

```
plt.title("Кластеризация OPTICS (исходные данные)")
```

```
plt.xlabel("X")
```

```
plt.ylabel("Y")
```

```
# --- Визуализация 2: График достижимости ---
```

```
plt.subplot(1, 2, 2)
```

```
space = np.arange(len(X))
```

```
plt.plot(space[reachability.argsort()], np.sort(reachability), 'k.', markersize=2)
```

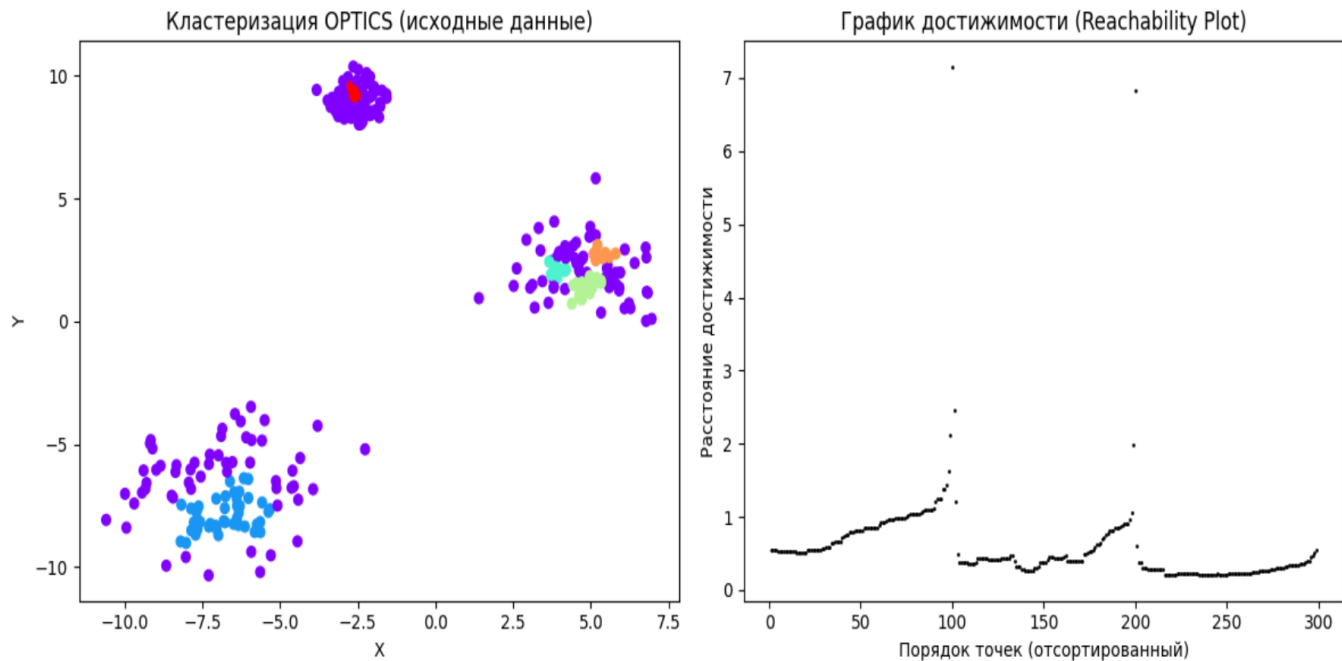
```
plt.title("График достижимости (Reachability Plot)")
```

```
plt.xlabel("Порядок точек (отсортированный)")
```

```
plt.ylabel("Расстояние достижимости")
```

```
plt.tight_layout()
```

```
plt.show()
```



Объяснение кода:

1. Мы создаем данные с тремя кластерами, имеющими разный разброс (плотность).
2. Обучаем OPTICS с параметром `max_eps=np.inf`, чтобы алгоритм сам определил структуру.
3. Строим 2 графика:

Слева — сами точки, окрашенные согласно найденным кластерам.

Справа — **график достижимости**. "Впадины" на этом графике соответствуют кластерам (чем ниже значение, тем плотнее область). "Пики" — это границы между кластерами или шум.

3. Задачи для проверки модели (с объяснением)

Задача 1: Анализ влияния `min_samples`

Условие: При использовании OPTICS параметр `min_samples` определяет минимальное количество точек в окрестности для формирования ядра кластера. Что произойдет с графиком достижимости, если сильно увеличить `min_samples` (например, с 5 до 50)?

Как это повлияет на количество точек, помеченных как шум?

Ответ/Объяснение: График достижимости: Станет более сглаженным. Расстояния достижимости увеличатся, так как точкам будет сложнее найти 50 соседей.

Количество шума: Увеличится. Кластеры будут определяться только в очень плотных областях, редкие области станут шумом.

Задача 2: Извлечение DBSCAN-подобных кластеров

Условие: Модель OPTICS обучена. У нас есть новый радиус $\text{new_eps} = 0.8$. Как получить разметку, эквивалентную DBSCAN с этим радиусом, без переобучения всей модели?

Программный код:

```
from sklearn.cluster import cluster_optics_dbscan

# Предположим, модель 'clust' уже обучена на данных X

# Извлекаем кластеры для  $\text{eps} = 0.8$ 

dbscan_labels = cluster_optics_dbscan(
    reachability=clust.reachability_,
    core_distances=clust.core_distances_,
    ordering=clust.ordering_,
    eps=0.8
)

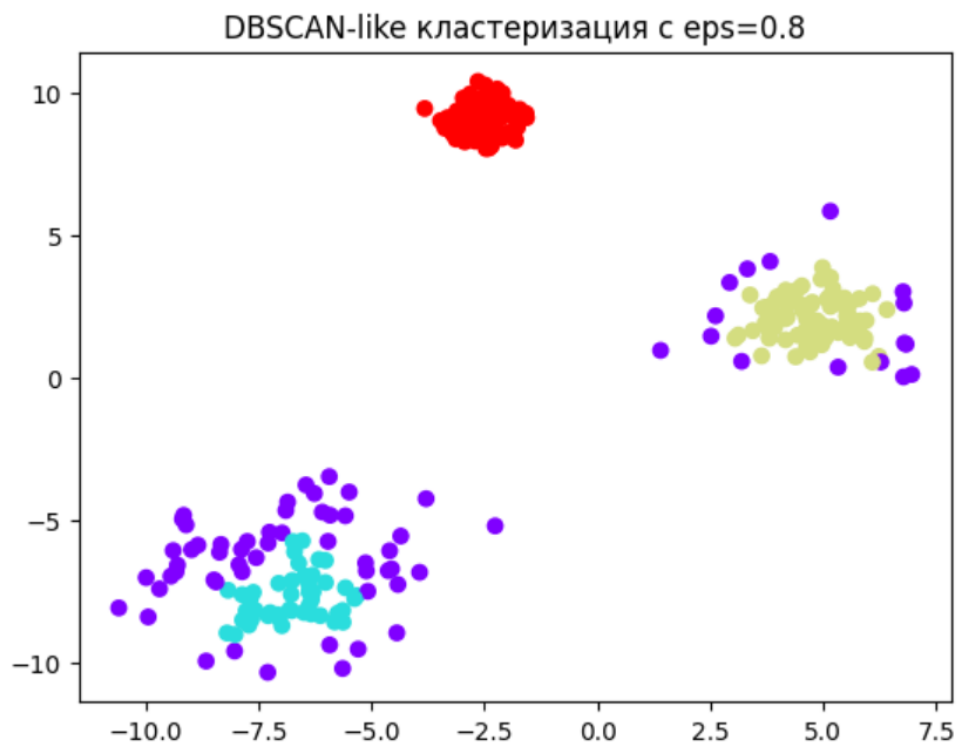
# Визуализация результата

plt.scatter(X[:, 0], X[:, 1], c=dbscan_labels, cmap='rainbow')

plt.title(f'DBSCAN-like кластеризация с  $\text{eps}=0.8$ ')

plt.show()
```

...



Объяснение:

Функция `cluster_optics_dbscan` использует уже рассчитанные расстояния (`reachability_` и `core_distances_`) для мгновенного построения результата. Это линейная операция, не требующая повторного вычисления попарных расстояний.

Задача 3: Сравнение с DBSCAN (периферийные точки)

Условие: В тексте сказано, что "первые образцы каждой плотной области, обработанные OPTICS, имеют большое значение достижимости... и поэтому иногда будут помечены как шум".

- Приведите пример сценария, где DBSCAN пометит точку как граничную, а OPTICS — как шум.

Ответ/Объяснение:

Рассмотрим плотное ядро кластера и его разреженную "опушку".

1. **DBSCAN:** Начинает с любой точки. Если точка на опушке достижима из ядра (в радиусе ϵ), она станет граничной.
2. **OPTICS:** Строит упорядоченный список. Первая точка, с которой начинается обход нового кластера (например, случайная точка на опушке), часто не имеет "предыдущей" точки в кластере, поэтому её расстояние достижимости условно велико (или бесконечно). При извлечении кластеров (особенно методом χ_i) такие стартовые точки могут быть отсечены как шум, даже если они находятся рядом с кластером.